HANDS-ON WORKSHEET — Regularization in CNN

Session 8 – 25/10/2026 · Artificial Intelligence and Deep Learning · PhD Program, FISI–UNMSM

Student: Luis Alberto Alarcón Loayza

Results recorded from the notebook Regularization_CNN_Luis_Alarcon.ipynb executed in Google Colab (random seed 42).

Record your results

Dataset: MNIST. Training: 3,000 images · Validation: 1,000 images · Test: 10,000 images. 10 epochs, batch size 64, random seed 42.

Model	Final train accuracy	Best validation accuracy	Test accuracy	Observation
A. Baseline	0.9960	0.9770 (epoch 6)	0.9638	Highest training accuracy and largest train–validation gap (2.5 points). Validation accuracy fell from 0.977 (epoch 6) to 0.971 (epoch 10): signs of overfitting.
B. Dropout	0.9777	0.9750 (epoch 10)	0.9691	Gap reduced to 0.27 points; still improving at the last epoch and highest final validation accuracy (0.975). Better test accuracy than the baseline.
C. Augmentation + Dropout	0.9597	0.9710 (epoch 9)	0.9709	Lowest training accuracy, but best test accuracy, lowest test loss (0.0859) and lowest final validation loss (0.0782).

Which model achieved the highest training accuracy?

Answer: Model A (Baseline), with a final training accuracy of 0.9960 (99.60%).

Which model achieved the highest validation accuracy?

Answer: Model A (Baseline) reached the highest best validation accuracy, 0.977 at epoch 6, but it did not keep it: by epoch 10 its validation accuracy had dropped to 0.971. In the last epoch, Model B (Dropout) had the highest validation accuracy (0.975), compared with 0.971 for the Baseline and 0.969 for Augmentation + Dropout. On a validation set of 1,000 images, these differences correspond to only 2 to 6 images.

Which model achieved the highest test accuracy?

Answer: Model C (Augmentation + Dropout), with a test accuracy of 0.9709 (97.09%) and the lowest test loss (0.0859), followed by Dropout (0.9691) and the Baseline (0.9638).

Did any model show a large gap between training and validation accuracy? Explain.

Answer: Yes, the Baseline showed the largest gap: 0.9960 training accuracy versus 0.971 validation accuracy in the last epoch (2.5 percentage points). Its training loss fell almost to zero (0.0205), while its validation loss stayed much higher (0.0893), and its validation accuracy decreased after epoch 6. This is a typical sign of overfitting: the model kept improving on the 3,000 training images but stopped

improving on unseen images. Dropout reduced the gap to only 0.27 points, and with augmentation plus dropout the validation accuracy was even higher than the training accuracy (gap of −0.93 points). This negative gap does not mean the model is worse: the training accuracy of models B and C is measured while dropout and augmentation are active, which makes the training task harder, whereas validation is measured on the original images with all neurons active.

Questions for analysis

1. What is overfitting? Describe one sign of overfitting visible in a training curve.

Answer: Overfitting happens when a model learns the training data too closely, including its noise and specific details, instead of the general patterns. As a result, it performs very well on the training data but worse on new, unseen data. A visible sign in the curves: the training loss keeps decreasing (and the training accuracy keeps increasing) while the validation loss stops improving or starts to rise, so the gap between the training and validation curves grows with each epoch.

2. Why can a model have high training accuracy but lower validation accuracy?

Answer: Because training accuracy is measured on examples the model has already seen and used to update its weights, so the model can partly memorize them. The validation set contains images the model never used for learning, so it measures generalization. If the model memorizes specific details of the 3,000 training images (for example, particular handwriting styles) instead of the general shape of each digit, it will be less accurate on the validation images. A small training set and a model with many parameters (over 225,000 here) make this more likely.

3. What does dropout do during training? Is dropout normally active during inference?

Answer: During training, dropout randomly switches off a fraction of the neurons at each training step (in this worksheet, 40% of the units after the dense layer). Because no neuron can rely on specific other neurons always being present, the network learns more robust, distributed features and depends less on any single unit, which reduces overfitting. Dropout is not active during inference: all neurons are used when the model predicts on validation or test data. Keras handles this automatically, scaling the activations during training so that their expected values match at inference.

4. How does data augmentation create more variation without collecting new original images?

Answer: Data augmentation applies small random transformations to the existing training images every time they are shown to the model. In this worksheet, RandomTranslation(0.05, 0.05) shifts each digit by up to 5% horizontally and vertically, so in every epoch the model sees slightly different versions of the same 3,000 images. These variations are realistic, because a handwritten digit can appear slightly off-center, so the model learns that the identity of a digit does not depend on its exact position. The augmentation layer is only active during training; at inference the original images are used.

5. Why should the test set be kept separate from training and model selection?

Answer: The test set must give an unbiased estimate of how the final model will perform on truly new data. If we used it to train the model, or to choose between models and hyperparameters, information from the test set would leak into our decisions and the test accuracy would be optimistically biased. For that reason, the validation set is used for monitoring and model selection, and the test set is used only once, at the end.

6. Did regularization improve the results in your experiment? Support your answer with your recorded values.

Answer: Yes, regularization improved generalization in this experiment, although the improvement is moderate because MNIST is a relatively easy dataset. Test accuracy increased from 0.9638 (Baseline) to 0.9691 (Dropout) and 0.9709 (Augmentation + Dropout), a gain of 0.71 percentage points, which means 71 more correctly classified images out of 10,000. Test loss decreased from 0.1190 to 0.0984 and 0.0859, about 28% lower. The train–validation gap shrank from 2.5 points (Baseline) to 0.27 points (Dropout) and to −0.93 points (Augmentation + Dropout), and the final validation loss was lowest for Augmentation + Dropout (0.0782, compared with 0.0893 for the Baseline). Training accuracy decreased with regularization (0.9960 → 0.9777 → 0.9597), but this is expected, because regularization makes the training task harder in order to prevent memorization. The best validation accuracy alone did not show a clear improvement (0.977, 0.975 and 0.971), but the Baseline lost its peak in later epochs, and these differences are only 2 to 6 images out of 1,000; the test set, with 10,000 images, gives a more reliable comparison. These results come from a single run with seed 42, so small differences could change with another seed.

Optional extension — Early stopping

Epochs completed: 11 (out of a maximum of 20)

Best validation loss: 0.0906 (reached at epoch 9)

What does this tell you?

Answer: The dropout model reached its lowest validation loss (0.0906) at epoch 9. After that, the training accuracy kept increasing (from 0.9727 at epoch 9 to 0.9840 at epoch 11), while the validation accuracy decreased (from 0.9710 to 0.9640) and the validation loss did not improve (0.0949 and 0.0943). This is the point where the model starts to overfit. Because the validation loss did not improve for two consecutive epochs (patience = 2), training stopped automatically at epoch 11 instead of 20, and the weights from epoch 9 were restored. Early stopping therefore acts as another form of regularization: it selects the number of epochs automatically from the validation data, prevents the model from memorizing the training set, and saves training time.